

Perturbed Derivative Decoding Based on Ordered Statistics for Cyclic Codes

Shuyan Yu^{ID}, Tianyu Li^{ID}, and Qin Huang^{ID}, *Senior Member, IEEE*

Abstract—It has been shown that perturbations can improve the performance of *ordered statistics decoding* (OSD). In this paper, we focus on perturbations for *derivative decoding based on OSD* (DD-OSD) for cyclic codes, where the results of OSD in different derivative directions vote for the estimate of codewords. It reveals that the *local maximum likelihood decoding* (*l*-MLD) errors in derivative directions may trap DD-OSD. It proves that such traps can be detected by the null space of the derivative ascendants of cyclic codes, thereby guiding us to perform perturbations to avoid the traps. Simulation results show that perturbed DD-OSD with order-1 can achieve 1.3 dB gain over OSD with order-4 for extended BCH codes, and perform closely to maximum likelihood decoding with moderate complexity.

Index Terms—Ordered statistics decoding, derivative decoding, perturbation.

I. INTRODUCTION

ORDERED *statistics decoding* (OSD) is a general decoding algorithm that offers asymptotically optimal performance for linear codes of short to medium length [1]. In OSD, the received bits and generator matrix are re-ordered according to reliability. Then the re-ordered generator matrix is transformed into the systematic form using Gaussian elimination. This places the *most reliable independent positions* (MRIPs) into the information positions. The union of MRIPs and the most reliable positions beside MRIPs is referred to as the control band. The information bits and *test error patterns* (TEPs) are added to be re-encoded, and the most likely candidate codeword is output as the decoding result.

For OSD, errors in MRIPs and *maximum likelihood decoding* (MLD) errors determine the performance. On the one hand, the errors in MRIPs are exhaustively tested using TEPs across various orders. As the decoding order increases, the performance of OSD improves. For the linear code $\mathcal{C}(n, k)$ with length n , dimension k and minimum distance d , OSD with order $l = \lfloor d/4 - 1 \rfloor$ can asymptotically approach the performance of MLD. Thus, many important studies focus on reducing the complexity of high-order OSD or achieving

better performance of low-order OSD [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17]. It shows in [8] that perturbations or multiple random bases on MRIPs can enhance the performance of OSD by generating multiple information sets, thereby increasing the likelihood of selecting one with fewer errors for correct re-encoding. On the other hand, when MLD errors occur, it leads directly to the decoding failure.

Recently, *derivative decoding based on ordered statistics decoding* (DD-OSD) has been proposed to significantly enhance OSD for extended cyclic codes [18], [19]. It consists of three steps and runs in an iterative manner. First, it calculates the *log-likelihood-ratio* (LLR) of *minimal derivative descendants* (mDDs) in various directions. Then, it decodes these mDDs by OSD. Finally, the mDD decoding estimates are used to vote for the estimate of the original cyclic code. Due to the small dimension of mDDs and the voting scheme, it can outperform OSD with higher-order for extended cyclic codes.

As MLD errors lead to the failure of OSD, this paper demonstrates that the *local MLD* (*l*-MLD) errors in derivative directions may trap the decoder of DD-OSD. In particular, we prove that *l*-MLD errors in derivative directions can result from a wrong estimate in DD-OSD that is not a legal codeword of the original cyclic code, but belongs to its *cyclic derivative ascendant* (cDA). In other words, we can detect such decoding traps by the null space of the original code and its cDA. Once a trap is detected, we propose to escape from the current trapped decoding and restart the decoding with perturbations on the control band. It may move errors out of MRIPs in mDDs, thus helping to avoid the trap.

In order to make perturbations efficient, OSD usually selects bits with higher reliabilities from the control band for perturbations. However, we propose perturbing bits with lower reliabilities in the control band for derivative decoding. Since a derivative bit in an mDD depends on two bits of the original codeword, its reliability is largely determined by the original bit with lower reliability. Perturbing the bits with lower reliabilities is more likely to cause the derivative bits to be perturbed, thereby reducing the reliabilities of errors. As a result, such perturbations can effectively move errors out of MRIPs in mDDs, significantly improving the performance of these perturbed mDDs.

Simulation results show that the proposed *perturbed DD-OSD* (PDD-OSD) with order-1 achieves 1.3 dB gain over OSD with order-4 for *extended Bose-Chaudhuri-Hocquenghem*

Received 31 August 2025; revised 21 December 2025 and 5 March 2026; accepted 16 March 2026. Date of publication 24 March 2026; date of current version 3 April 2026. This work was supported by the National Natural Science Foundation of China under Grant 62331002. An earlier version of this paper was presented in part at the 2025 IEEE Information Theory Workshop (IEEE ITW 2025) [DOI: 10.1109/ITW62417.2025.11240295]. The associate editor coordinating the review of this article and approving it for publication was R. Thobaben. (Corresponding author: Qin Huang.)

The authors are with the School of Electronic and Information Engineering, Beihang University, Beijing 100191, China (e-mail: yushuyan@buaa.edu.cn; tianyu@buaa.edu.cn; qhuang.smash@gmail.com).

Digital Object Identifier 10.1109/TCOMM.2026.3677154

(eBCH) codes and performs closely to MLD with moderate complexity. It also outperforms DD-OSD with the same order by about 1.2 dB, while requiring a closely similar average complexity.

The organization of this paper is as follows. Section II reviews OSD and DD-OSD. In Section III, we analyze the decoding traps in DD-OSD, the perturbations in DD-OSD and details the proposed PDD-OSD algorithm. The simulation results and complexity analysis are given in Section IV. In Section V, we draw a conclusion.

II. PRELIMINARY

A. OSD

We consider a binary linear code $\mathcal{C}(n, k)$ with dimension k and length n . Let $\mathbf{G}$ denote its generator matrix and $\mathbf{c} = [c_i, i \in [n]]$ denote a codeword. Here, we set $[n] := \{0, 1, \dots, n-1\}$. The code is used for transmission over an *additive white Gaussian noise* (AWGN) channel with *binary phase shift keying* (BPSK) modulation. The received vector is denoted by $\mathbf{r} = [r_i, i \in [n]]$, and its hard decision version is $\mathbf{y} = [y_i, i \in [n]]$. The LLR vector is $\mathbf{L} = [L_i, i \in [n]]$. Here, L_i for each bit is given by:

$$L_i = \ln \frac{\Pr(r_i | c_i = 0)}{\Pr(r_i | c_i = 1)},$$

where $\Pr(\cdot)$ is the probability function.

In OSD, the received vector $\mathbf{r}$ is re-ordered in descending order of LLR magnitude. Let $\mathbf{r}' = \Pi_1(\mathbf{r})$ denote the re-ordered vector, where Π_1 is the sorting permutation. The generator matrix $\mathbf{G}$ is also re-ordered, i.e., $\mathbf{G}' = \Pi_1(\mathbf{G})$. If the first k columns of $\mathbf{G}'$ are not linearly independent, a second permutation Π_2 is used to keep independence, i.e., $\mathbf{G}'' = \Pi_2(\Pi_1(\mathbf{G}'))$. And it can be converted into the systematic form via Gaussian elimination. The received and hard-decision vectors are updated accordingly $\mathbf{r}'' = \Pi_2(\Pi_1(\mathbf{r}'))$, $\mathbf{y}'' = \Pi_2(\Pi_1(\mathbf{y}'))$.

Then, hard decisions in MRIPs produce an information vector $\tilde{\mathbf{u}} = [\tilde{u}_i]$ using $\tilde{u}_i = \mathbb{1}[r''_i < 0]$, where $\mathbb{1}[\cdot]$ is the indicator function. For OSD with order- l , all TEPs of weight up to l are generated. By adding these to $\tilde{\mathbf{u}}$, the new information vectors are obtained and then re-encoded to generate candidate codewords $\mathbf{c}^{(j)}$, with $0 \leq j < \sum_{i=0}^l \binom{k}{i}$. The decoder selects the codeword minimizing the weighted Hamming distance:

$$d(\mathbf{y}'', \mathbf{c}^{(j)}) = \sum_{i: y''_i \neq c^{(j)}_i} |L_i|.$$

The codeword with minimum distance is denoted by $\tilde{\mathbf{c}}_{\text{opt}}$. The stopping rules may be applied to avoid evaluating unlikely candidates. The inverse permutations are applied to obtain the decoding result $\hat{\mathbf{c}}_{\text{opt}} = \Pi_1^{-1}(\Pi_2^{-1}(\tilde{\mathbf{c}}_{\text{opt}}))$.

Furthermore, the performance of OSD is determined by errors in MRIPs and MLD error. The error probability of OSD, denoted by P_{OSD} , is bounded by

$$P_{\text{OSD}} \leq P_{\text{List}} + P_{\text{ML}}, \quad (1)$$

where P_{List} is the probability that the number of errors in MRIPs exceeds the OSD order, and P_{ML} is the probability

of MLD errors. According to (1), the errors in MRIPs and the MLD errors determine the performance of OSD. In terms of errors in MRIPs, introducing perturbations [8], [20] is one of effective ways to reduce these errors, thus improving the performance of OSD. It is shown that multiple information sets are generated through perturbations. It is possible to obtain an information set by perturbations that can be re-encoded to a correct codeword.

MLD errors refer to the case where OSD still outputs an incorrect codeword with smaller weighted Hamming distance, even when the number of errors is less than or equal to l . In this case, there exists an incorrect candidate re-encoded codeword $\mathbf{c}^{(j')}$, $0 \leq j' \leq \sum_{i=0}^l \binom{k}{i}$, such that

$$d(\mathbf{y}'', \mathbf{c}^{(j)}) \geq d(\mathbf{y}'', \mathbf{c}^{(j')}),$$

where $0 \leq j \leq \sum_{i=0}^l \binom{k}{i}$ and $j \neq j'$. In a special case where the hard-decision input to OSD is a codeword, we have $j' = 0$. Since the candidate codeword $\mathbf{c}^{(j')}$ is not the correct one, an MLD error occurs. If an MLD error occurs, it leads directly to the decoding failure.

B. DD-OSD

The derivative decoding scheme for cyclic codes and extended cyclic codes is proposed in [18] and [19], which improves the performance of OSD. Cyclic codes form a well-known class of linear codes with a cyclic structure. This class includes BCH codes [21], [22], Reed-Muller (RM) codes [23], [24], and others. The extended cyclic codes are formed by adding an overall parity-check bit to the codewords of cyclic codes.

Consider the extended cyclic codes $\mathcal{C}(n, k)$ with length n and dimension k . The derivative decoding scheme proposes to decode $\mathcal{C}(n, k)$ based on their mDDs in different directions. The mDDs are defined as the linear subspace containing the derivatives of all codewords in different directions with the smallest dimension. Their dimension is the same in different directions and much smaller than that of the original codes [18], [25]. For example, for the (128, 36) eBCH code, its mDDs in different directions are a (64, 14) linear code. Moreover, by analyzing the Mattson-Solomon polynomials and their exponent sets [26], [27], it has been proven that the mDDs in different directions are equivalent codes. As a result, the derivative decoding is performed using a single decoder for the mDDs in different directions.

Specifically, we first introduce the notation used in the derivative decoding. Let $\mathbb{F}_{2^m}$ denote the finite field of 2^m elements. Let α denote a primitive element of $\mathbb{F}_{2^m}$. Let α^∞ denote the additive identity in $\mathbb{F}_{2^m}$. The codeword coordinates are indexed by the elements of $\mathbb{F}_{2^m}$. Let $\mathbf{c} = [c_{\alpha^i}, i \in I]$ denote a codeword of $\mathcal{C}(n, k)$, where $I = \{[n-1] \cup \{\infty\}\}$. Each derivative direction is represented by an element $\beta \in \mathbb{F}_{2^m}$, and the set of all derivative directions is denoted by B . The derivative of a given codeword $\mathbf{c} = [c_{\alpha^i}, i \in I]$ in the direction β is defined as

$$\mathbf{c}^\beta = [c_{\alpha^i}^\beta, i \in I] = [c_{\alpha^{i+\beta}} + c_{\alpha^i}, i \in I].$$

Instead of directly decoding $\mathbf{c}$, the derivative decoding applies effective decoders to the derivative codewords $\mathbf{c}^\beta$ for $\beta \in B$.

DD-OSD decodes c^β by OSD in the derivative directions. It consists of three steps and works in an iterative manner. First, it calculates the derivative LLR vector of mDDs in various directions. Then, it decodes these mDDs by OSD. Finally, the mDD decoding estimates are used to vote for the estimate of the original extended cyclic code.

Consider transmitting a codeword c of $\mathcal{C}$ over an AWGN channel with BPSK modulation. In particular, DD-OSD first calculates the derivative LLR vector in the direction β as follows:

$$\mathbf{L}^\beta = [L_{\alpha^i}^\beta, i \in I], \quad (2)$$

by taking

$$L_{\alpha^i}^\beta = 2 \tanh^{-1} \left(\tanh \left(\frac{L_{\alpha^i}}{2} \right) \tanh \left(\frac{L_{\alpha^j}}{2} \right) \right),$$

where $\alpha^j = \alpha^i + \beta$. Note that the cyclic shift of derivative LLR vector $\mathbf{L}^\beta$ is equal to the derivative of the cyclic shift of $\mathbf{L}$. It allows us to use one of LLR vectors and its cyclic shift to calculate the derivative LLR vectors in different directions.

Then, in the direction β , it decodes $\mathbf{L}^\beta$ based on OSD and obtains an estimate $\hat{c}^\beta = [\hat{c}_{\alpha^i}^\beta, i \in I]$. The soft vote is given by

$$\tilde{\mathbf{L}}^\beta = [(1 - 2\hat{c}_{\alpha^i}^\beta)L_{\alpha^i+\beta}, i \in I]. \quad (3)$$

Finally, the decoding estimate of $\mathcal{C}$ is voted by soft votes from the set of directions, i.e.,

$$\tilde{\mathbf{L}} = \frac{1}{|B|} \sum_{\beta \in B} \tilde{\mathbf{L}}^\beta. \quad (4)$$

Here, $\sum$ is the summation by components. The algorithm outputs the hard decision on $\tilde{\mathbf{L}}$ if it is a codeword of $\mathcal{C}$. Otherwise, take $\mathbf{L} = \tilde{\mathbf{L}}$ and go for a new iteration until reaching a maximum iteration number.

The voting scheme uses the decoding results in different directions to obtain the final decoding result of original code. It allows for tolerating the decoding errors in some directions. The voting scheme and the smaller dimension of mDDs both lead to a significant improvement on OSD. It shows that DD-OSD with order-1 based on mDDs can outperform OSD with order-3.

III. PERTURBATIONS FOR DD-OSD

In this section, we first investigate the occurrence and detection of decoding traps that result from l -MLD errors in DD-OSD. Such traps can be detected by the null space of the original code and its cDA. For the detectable decoding traps, we propose to escape from the current trapped decoding and restart the decoding with perturbations on the control band to avoid the traps. We present a perturbation method that effectively moves errors out of MRIPs in mDDs, thereby significantly improving their performance. Finally, the proposed PDD-OSD algorithm is presented based on the detection of decoding traps and perturbations.

A. Decoding Trap in DD-OSD

Recall that an MLD error directly leads to OSD decoding failure, even when the number of errors does not exceed the order l . Moreover, such errors cannot be mitigated by applying perturbations or by refining the information set in other ways. In the following, we investigate the occurrence of decoding traps caused by l -MLD errors in DD-OSD. We regard the MLD error that occurs in a single OSD of DD-OSD, as an l -MLD error. We prove that the sufficient conditions for this decoding trap are as follows: the voted estimate of DD-OSD is not a legal codeword of the original cyclic code, but belongs to its cDA. This enables the detection of decoding traps during the decoding process.

Before presenting the next theorem that investigates the occurrence of such decoding traps, we briefly recall key definitions from [18] and [19]. The *cyclic derivative descendant* (cDD), denoted by $\mathcal{D}(\mathcal{C})$, is the extended cyclic code with the smallest dimension which contains all the derivative codewords of $\mathcal{C}$ and consists of mDDs. The cDA of cDDs, denoted by $\mathcal{A}(\mathcal{D}(\mathcal{C}))$, is the extended cyclic code with the largest dimension such that its derivative codewords belong to cDDs. Considering the w -th decoding iteration of DD-OSD, let $\mathbf{L}^{(w)} = [L_{\alpha^i}^{(w)}, i \in I]$ denote the updated LLR vector and $\mathbf{c}^{(w)} = [c_{\alpha^i}^{(w)}, i \in I]$ denote its hard decision, respectively.

Theorem 1: Consider an extended cyclic code $\mathcal{C}$. A voted estimate $\mathbf{c}^{(w)} \in \mathcal{A}(\mathcal{D}(\mathcal{C})) \setminus \mathcal{C}$ leads to a decoding trap in DD-OSD.

Proof: Consider $\mathbf{c}^{(w)} \in \mathcal{A}(\mathcal{D}(\mathcal{C})) \setminus \mathcal{C}$, i.e., the output of the w -th decoding iteration of DD-OSD is not a legal codeword of the original cyclic code but belongs to its cDA. Then, in the $(w+1)$ -th decoding iteration, the derivative LLR vector $\mathbf{L}^\beta$ is calculated according to (2) for the direction β , and derivative vector $\mathbf{c}^\beta$ is the hard decision of $\mathbf{L}^\beta$. When $\mathbf{c}^\beta$ is a codeword of mDDs, the output of OSD in the direction β must be $\mathbf{c}^\beta$, which is a l -MLD error in an mDD. For the bit at coordinate α^i , the soft votes from the direction set B are

$$\begin{aligned} L_{\alpha^i}^{(w+1)} &= \sum_{\beta \in B} \left(1 - 2 c_{\alpha^i}^\beta \right) \cdot L_{\alpha^i+\beta}^{(w)} \\ &= \sum_{\beta \in B} \left[1 - 2 \left(c_{\alpha^i}^\beta \oplus c_{\alpha^i+\beta}^{(w)} \right) \right] \cdot \left| L_{\alpha^i+\beta}^{(w)} \right| \\ &= \sum_{\beta \in B} \left[1 - 2 \left(c_{\alpha^i}^{(w)} \oplus c_{\alpha^i+\beta}^{(w)} \oplus c_{\alpha^i+\beta}^{(w)} \right) \right] \cdot \left| L_{\alpha^i+\beta}^{(w)} \right| \\ &= \sum_{\beta \in B} \left[1 - 2 \left(c_{\alpha^i}^{(w)} \right) \right] \cdot \left| L_{\alpha^i+\beta}^{(w)} \right|. \end{aligned}$$

The soft vote from the direction β for the bit at coordinate α^i is determined by $c_{\alpha^i}^\beta$ and $L_{\alpha^i+\beta}^{(w)}$, for $i \in I$. The value of soft votes $|L_{\alpha^i}^{(w+1)}|$ is the sum of the absolute values of LLRs from all other positions. The hard decision of $L_{\alpha^i}^{(w+1)}$ is still $c_{\alpha^i}^{(w)}$. Thus, the hard decision output of the $(w+1)$ -th decoding iteration is also $\mathbf{c}^{(w)} = [c_{\alpha^i}^{(w)}, i \in I]$, which remains unchanged. As a result, DD-OSD fails to escape the incorrect decision, becoming stuck in a decoding trap. ■

Based on the above theorem, it is shown that when the l -MLD errors result from a codeword that belongs to cDA

rather than the original code, DD-OSD is trapped. In other words, we can detect such decoding traps by the null spaces of the original code and its cDA, while $\mathcal{A}(\mathcal{D}(\mathcal{C}))$ can be calculated in advance using the known $\mathcal{C}$. The detection results guide us to escape from the current trapped decoding and restart the decoding with perturbations on the control band, thereby avoiding the decoding traps.

B. Perturbations for DD-OSD

Consider the received LLR vector $\mathbf{L} = [L_{\alpha^i}, i \in I]$. Let $\mathbf{L}' = [L'_{\alpha^i}, i \in I]$ denote the re-ordered LLR vector. The perturbed control band is referred to as the first k' most reliable positions, where $k' > k$. Let $L_{\alpha^{i_0}}, L_{\alpha^{i_1}}, \dots, L_{\alpha^{i_{k'-1}}}$ denote the LLR components of the original LLR vector corresponding to the control band. Their coordinates are denoted by $\mathcal{M} = \{\alpha^{i_0}, \alpha^{i_1}, \dots, \alpha^{i_{k'-1}}\}$. And a map $\mathcal{F}$ is defined to relate the coordinates before and after sorting, i.e.,

$$\mathcal{F} : \{\alpha^\infty, \alpha^0, \alpha^1, \dots, \alpha^{n-2}\} \rightarrow \{\alpha^{i_\infty}, \alpha^{i_0}, \alpha^{i_1}, \dots, \alpha^{i_{n-2}}\}. \quad (5)$$

It satisfies $f(\alpha^j) = \alpha^{i_j}$ for $j \in I$.

For each perturbation, the specific perturbed positions can be determined by a perturbed index set, which is defined as follows:

Definition 1: For each perturbation, the perturbed index set is defined as the set of positions in the perturbed control band whose LLR values are scaled down, i.e.,

$$\mathcal{I} = \{i'_0, i'_1, \dots, i'_{s-1}\}.$$

For $i \in \mathcal{I}$, L'_i denotes the LLR value of the i -th position in the perturbed control band after sorting. The size of $\mathcal{I}$ is s .

Specifically, the perturbed index set is constructed from the components in the control band whose positions belong to $\mathcal{I}$. Given a perturbed index set $\mathcal{I} = \{i'_0, i'_1, \dots, i'_{s-1}\}$, the perturbed LLR components in $\mathbf{L}'$ are $L'_{i'_0}, L'_{i'_1}, \dots, L'_{i'_{s-1}}$. We have $\{L'_{i'_0}, \dots, L'_{i'_{s-1}}\} = \{L'_{\alpha^{i_{j_0}}}, \dots, L'_{\alpha^{i_{j_{s-1}}}}\}$, where $\{\alpha^{i_{j_0}}, \dots, \alpha^{i_{j_{s-1}}}\}$ is a subset of $\{\alpha^{i_0}, \alpha^{i_1}, \dots, \alpha^{i_{k'-1}}\}$.

Then, we need to identify the corresponding coordinates of perturbed LLR components in $\mathbf{L}$ by an inverse map, denoted by $\mathcal{F}^{-1}$. The set of corresponding coordinates in the original LLR vector is denoted by $\mathcal{P}$, i.e.,

$$\mathcal{P} = \{\mathcal{F}^{-1}(i), i \in \{\alpha^{i_{j_0}}, \dots, \alpha^{i_{j_{s-1}}}\}\}. \quad (6)$$

For each perturbation, the LLR values at the positions in $\mathcal{P}$ are scaled by a factor $0 < \lambda < 1$, thereby moving errors out of MRIPs.

Given p different perturbations, there are p different perturbed index sets $\mathcal{I}_0, \mathcal{I}_1, \dots, \mathcal{I}_{p-1}$ with sizes $s_0, s_1, \dots, s_{p-1}$, respectively. Leveraging different perturbations, we tentatively reduce the reliabilities of bits in various perturbed positions, thereby lowering the reliabilities of errors.

Here, we present a function `Perturbation` to introduce a perturbation, with the pseudo code given in Function 1. In this function, we first sort the LLR vector based on the magnitude in decreasing order to obtain the set $\mathcal{M}$ and the map $\mathcal{F}$. Then, we obtain the perturbed coordinate set $\mathcal{P}$ based on (6). Finally, we obtain the perturbed LLR vector $\bar{\mathbf{L}}$ with λ and $\mathcal{P}$.

Function 1 Perturbation

Input: The LLR $\mathbf{L}$, the perturbed index set $\mathcal{I}$, the perturbed factor λ

Output: The perturbed LLR $\bar{\mathbf{L}}$

- 1: $[\mathcal{M}, f] \leftarrow |L_{\alpha^{i_\infty}}| > |L_{\alpha^{i_0}}| > \dots > |L_{\alpha^{i_{n-2}}}|$
- 2: Get $\mathcal{P}$ based on (6) with $\mathcal{I}$ and f
- 3: Get $\bar{\mathbf{L}}$ with $\mathcal{P}$ and λ

C. Effective Perturbations

In this subsection, we first present the lemma for selecting the index sets of perturbations to effectively move errors out of MRIPs for mDDs. It reveals that within the specified range, a lower reliability of the perturbed bit leads to a greater number of perturbed mDDs, whose reliabilities of erroneous bits are reduced. Then, we further analyze the error probability of mDDs with effective perturbations.

Consider the direction β and OSD with order l in an mDD. Let $k_{\mathcal{M}}$ denote the dimension of mDDs. Let $\{i'_{e_0}, \dots, i'_{e_h}\}$ denote the perturbed errors index set with coordinates $\{\alpha^{i'_{e_0}}, \dots, \alpha^{i'_{e_h}}\}$ in $\mathbf{L}$. Note that $\{i'_{e_0}, \dots, i'_{e_h}\} \subset \mathcal{I}$. If the corresponding bits in $\mathcal{I}$ are all correct, then $\{i'_{e_0}, \dots, i'_{e_h}\}$ is an empty set.

Lemma 1: Consider the index set of perturbation $\mathcal{I} = \{i'_0, i'_1, \dots, i'_{s-1}\}$. To move the error bits out of MRIPs via a perturbation, it is required to satisfy that

$$\max\{i'_0, i'_1, \dots, i'_{s-1}\} \leq n/2 + k_{\mathcal{M}}.$$

Among the perturbed bits corresponding to $\mathcal{I}$, the error bits with a non-empty index subset $\{i'_{e_0}, \dots, i'_{e_h}\}$ lead to a reduction of the error reliabilities in at least $\max\{i'_{e_0}, \dots, i'_{e_h}\} - 1$ mDDs.

Proof: In this proof, we first show the valid range for selecting the index set to move errors out of MRIPs of mDDs. Then, we present perturbing low-reliability error bits within this range leads to a larger number of mDDs to be perturbed.

Consider an arbitrary perturbed bit with index $i'_{j_q} \in \mathcal{I}$ and coordinate α^{j_q} . Suppose that $i'_{j_q} > n/2 + k_{\mathcal{M}}$. In this case, at most $n/2 - k_{\mathcal{M}} - 1$ bits have smaller amplitude values. Equation (2) can be simplified based on min-sum approximation [28], i.e.,

$$L_{\alpha^i}^\beta = \text{sign}(L_{\alpha^i})\text{sign}(L_{\alpha^j}) \min\{|L_{\alpha^i}|, |L_{\alpha^j}|\},$$

where $\text{sign}(\cdot)$ denotes the sign function and $\alpha^j = \alpha^i + \beta$. Under this approximation, the reliability of a derivative bit is determined by the original bit with a lower reliability.

Thus, the number of bits in mDDs whose amplitudes are smaller than $|L_{\alpha^{j_q}}^\beta|$ is less than $n/2 - k_{\mathcal{M}}$. In other words, the perturbed bit is already outside MRIPs in mDDs even before introducing perturbations, and the perturbations do not affect the errors in MRIPs of mDDs. As a result, to move the error bits out of MRIPs via a perturbation, the index set must satisfy $\max\{i'_0, i'_1, \dots, i'_{s-1}\} \leq n/2 + k_{\mathcal{M}}$.

After proving that the maximum index in the perturbed index set $\mathcal{I}$ cannot exceed $n/2 + k_{\mathcal{M}}$ to move errors out of MRIPs, next we focus on proving that perturbing error bits with lower reliabilities within this range can lead to more perturbed mDDs.

TABLE I
THE NUMBER OF DIRECTIONS WITH DIFFERENT ERRORS IN MRIPS

	$n_e \leq 1$	$n_e = 2$	$n_e = 3$
T_D	50	50	27
T_P	70	43	14

Consider the single error e_0 whose index is i'_{e_0} with coordinate α^{j_0} . Let T_{e_0} denote the set of directions in which the reliabilities of errors are reduced due to the perturbation caused by error e_0 . The perturbation of errors in mDDs depends on whether the derivative component is affected by the perturbed error e_0 . We have

$$T_{e_0} = \left\{ \beta : \tanh\left(\frac{L_{\alpha^{j_0}+\beta}}{2}\right) \tanh\left(\frac{L_{\alpha^{j_0}}}{2}\right) > \tanh\left(\frac{L_{\alpha^{j_0}+\beta}}{2}\right) \tanh\left(\frac{\lambda L_{\alpha^{j_0}}}{2}\right) \right\}. \quad (7)$$

Since the calculation of derivative LLR depends on the smaller term, it can be simplified as follows:

$$T_{e_0} = \{ \beta : |L_{\alpha^{j_0}+\beta}| > \lambda |L_{\alpha^{j_0}}| \}. \quad (8)$$

Note that $|L_{\alpha^{j_0}}|$ is the i'_{e_0} -th component in the ordering of LLR absolute values. Thus, it follows that $|T_{e_0}| \geq i'_{e_0} - 1$. For the set of perturbed errors index $\{i'_{e_0}, \dots, i'_{e_h}\}$, the set of perturbed directions is the union of the perturbed directions of each error, with a size of at least $\max\{i'_{e_0}, \dots, i'_{e_h}\} - 1$. ■

Lemma 1 reveals that among the first $n/2 + k_M$ positions, the perturbed bits with lower reliabilities lead to more mDDs where errors may be moved out of MRIPs. In addition, an example is given to compare the number of errors in mDDs before and after introducing a perturbation. Let $[a : b]$ denote the set of integers $\{a, a+1, \dots, b\}$, where $a \leq b$.

Example 1: Consider the (128, 36) eBCH code, whose mDDs are the (64, 14) linear code. Transmit the code over AWGN with BPSK. We set $|B| = 127$ and $l = 1$. For the given perturbed index set $\mathcal{I} = [20 : 25]$, there exists a perturbed error with index 24. It leads to a reduction in the error reliability in 40 directions. In more detail, we compare the errors in MRIPs for mDDs before and after introducing a perturbation. Let n_e denote the number of errors in MRIPs for mDDs. Table I gives the number of directions with different errors in MRIPs before and after introducing the perturbation $\mathcal{I}$, denoted by T_D and T_P , respectively.

From Table I, we observe that a larger number of mDDs contain at most one error after introducing the perturbation. It leads to more correct votes for the final decision, promising a good performance of low-order DD-OSD.

Then, we investigate the effect of an effective perturbation on reducing the error probability of mDDs. We consider the case in which one of the first $l+1$ errors is perturbed such that it is moved out of MRIPs in the mDDs. The following analysis focuses on evaluating the error probability of such a perturbed mDD.

Theorem 2: For an effective perturbed mDD, the error probability that the number of errors in the information positions exceeds the order l can be evaluated by

$$P'_{\text{List}}(l) = \sum_{j=l+1}^{n/2-k_M+l} Pr(\delta(l+2)) > \gamma(k_M-l+s-1) P(j) + \sum_{j=n/2-k_M+l+1}^{n/2} P(j), \quad (9)$$

where $\delta(i)$ is the i -th reliability value among error bits, $\gamma(i)$ is the i -th reliability value among correct bits, $Pr(\delta(l+2) > \gamma(k_M-l+s-1))$ is the probability that $\delta(l+2)$ exceeds $\gamma(k_M-l+s-1)$ given j errors, and $P(j)$ is the probability of containing j errors in the derivative vector of mDD.

Proof: To evaluate the error probability in MRIPs for such a perturbed mDD, we first analyze the values of erroneous and correct bits before a perturbation. Then, we analyze the effect of perturbation on these values to estimate the resulting error probability.

Given an effective perturbation on the direction β , let us consider the input as a Gaussian variable derived from two independent Gaussian random variables with mean μ and variance σ^2 , according to (2). Consider the all-zero codeword without loss of generality. Let $f_\beta(x)$ denote its probability density function (PDF). Based on Gaussian approximation [29], [30], $f_\beta(x)$ can be regarded as Gaussian PDF with mean $\mu' = \psi^{-1}\left(1 - \left(1 - \psi\left(\frac{2}{\sigma^2}\right)\right)^2\right)$ and variance $(\sigma')^2 = 2\mu'$. Here, $\psi(\mu)$ can be approximated by

$$\psi(\mu) = \begin{cases} e^{-0.4527\mu^{0.86}+0.0218}, & 0 < \mu < 10 \\ \sqrt{\pi/\mu} e^{-\mu/4} (1 - 10/7\mu), & \mu > 10. \end{cases}$$

Let $\Phi(z) = \int_{-\infty}^z \frac{1}{\sqrt{2\pi}} e^{-t^2/2} dt$ denote the standard Gaussian cumulative distribution function, and let $u(x)$ denote the unit step function. Note that, the derivative LLR is sorted based on reliability for the direction β . In this sorting, $\gamma(t_c)$ is t_c -th reliability value among correct bits, where t_c is an integer. Following [8], its PDF of $\gamma(t_c)$ is

$$f_{\gamma(t_c)}(x) = \frac{(n/2-j)!}{(t_c-1)!(n/2-j-t_c)!} \cdot (1-P_c(x))^{t_c-1} \cdot (P_c(x))^{n/2-t_c-j} \cdot f_c(x), \quad (10)$$

where $f_c(x) = \frac{1}{\Phi\left(\frac{\mu'}{\sigma'}\right)} \cdot \frac{1}{\sqrt{2\pi(\sigma')^2}} \cdot \exp\left(-\frac{(x-\mu')^2}{2(\sigma')^2}\right) \cdot u(x)$ and $P_c(x) = \frac{1}{\Phi\left(\frac{\mu'}{\sigma'}\right)} \cdot \left(\Phi\left(\frac{x-\mu'}{\sigma'}\right) + \Phi\left(\frac{\mu'}{\sigma'}\right) - 1\right) \cdot u(x)$. And $\delta(t_e)$ is t_e -th reliability value among error bits, where t_e is an integer. The PDF of $\delta(t_e)$ is

$$f_{\delta(t_e)}(x) = \frac{j!}{(t_e-1)!(j-t_e)!} \cdot (1-P_e(x))^{t_e-1} (P_e(x))^{j-t_e} \cdot f_e(x), \quad (11)$$

where $f_e(x) = \frac{1}{1-\Phi\left(\frac{\mu'}{\sigma'}\right)} \cdot \frac{1}{\sqrt{2\pi(\sigma')^2}} \cdot \exp\left(-\frac{(x+\mu')^2}{2(\sigma')^2}\right) \cdot u(x)$ and $P_e(x) = \frac{1}{1-\Phi\left(\frac{\mu'}{\sigma'}\right)} \cdot \left(\Phi\left(\frac{x+\mu'}{\sigma'}\right) - \Phi\left(\frac{\mu'}{\sigma'}\right)\right) \cdot u(x)$.

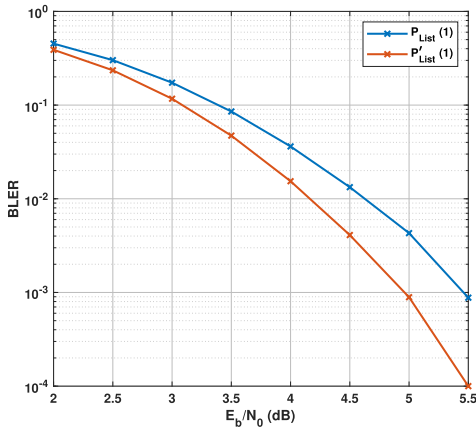


Fig. 1. The performance comparison of perturbed mDD before and after applying a perturbation in DD-OSD.

The probability that the value of the t_e -th erroneous bit $\delta(t_e)$ is greater than that of the t_c -th correct bit $\gamma(t_c)$ can be used to calculate $P_{\text{List}}(l)$ for β , i.e.,

$$Pr(\delta(t_e) > \gamma(t_c)) = \int_0^\infty f_{\gamma(t_c)}(x) \left(\int_x^\infty f_{\delta(t_e)}(y) dy \right) dx. \quad (12)$$

It is evaluated numerically using the trapezoidal rule for numerical integration. Thus, before applying a perturbation, we have $P_{\text{List}}(l)$ for the direction β as follows:

$$P_{\text{List}}(l) = \sum_{j=l+1}^{n/2-k_M+l} Pr(\delta(l+1) > \gamma(k_M-l)) P(j) + \sum_{j=n/2-k_M+l+1}^{n/2} P(j). \quad (13)$$

For a given effective perturbation with index set size s , we consider the case where one of the first $l+1$ errors is perturbed and moved out of MRIPs, such that the $(l+2)$ -th error becomes the new $(l+1)$ -th error with reliability value $\delta(l+2)$. Suppose that the other $s-1$ perturbed bits are all correct information bits and also moved out of information positions, which is the worst case. After the perturbation, the value of the k_M-l -th correct bit is $\gamma(k_M-l+s-1)$. The perturbed error probability $P'_{\text{List}}(l)$ can be evaluated by the probability that the value of the $l+2$ -th erroneous bit is greater than that of the $k_M-l+s-1$ -th correct bit, i.e., $Pr(\delta(l+2) > \gamma(k_M-l+s-1))$. As a result, we can obtain the perturbed error probability $P'_{\text{List}}(l)$ in (9). ■

Next, we provide an example to compare the values of $P_{\text{List}}(l)$ and $P'_{\text{List}}(l)$ for such a perturbed mDD.

Example 2: Consider the (128, 64) eBCH code, whose mDDs are the (64, 29) linear code. Transmit the code over AWGN with BPSK. Consider a single perturbed mDD. We set $s = 5$, $l = 1$ and $\lambda = 0.5$. In Fig. 1, we compare $P_{\text{List}}(1)$ and $P'_{\text{List}}(1)$ in different E_b/N_0 . It is shown that the performance of perturbed mDD can be effectively improved.

The above theorem and example show that, even in the worst case where $s-1$ correct bits in MRIPs are perturbed,

the reduction in the reliabilities of erroneous bits contributes more significantly to performance improvement.

Remark: Furthermore, based on the error probability evaluation, ineffective perturbations can also be reduced by avoiding those whose associated perturbed bits with higher reliabilities are all correct. Given the index set of perturbation $\mathcal{I} = \{i'_0, i'_1, \dots, i'_{s-1}\}$. For $i'_j \in \mathcal{I}$, its corresponding derivative bit is determined by the smaller term of original bits. To make the error in mDDs perturbed, the value of derivative bit needs to be determined by the perturbed bit in position i'_j . Thus, perturbing correct bits with high reliability should be avoided, since such perturbations have no impact on the derivative bits. According to the probability calculation in Theorem 2, we can compute $Pr(\gamma(\max(i'_0, i'_1, \dots, i'_{s-1})) < \delta(1))$ for the original codeword and set a threshold to reduce ineffective perturbations.

D. Perturbed DD-OSD

In this subsection, the details of PDD-OSD are presented, and its flowchart is shown in Fig. 2.

Consider transmitting a codeword c of the extended cyclic code $\mathcal{C}(n, k)$ over an AWGN channel using BPSK modulation. PDD-OSD is performed based on mDDs, while the decoding traps are detected based on cDA. The PDD-OSD decoding process consists of the following phases:

1) Perturbation: Given the perturbed index set $\mathcal{I}$ and the perturbed factor λ , perturbations are introduced to the original input LLR vector $\mathbf{L}$ according to Function 1, Perturbation, and obtain the perturbed LLR vector $\tilde{\mathbf{L}}$.

2) DD-OSD: The derivatives of $\mathcal{C}$ in different directions $\beta \in B$ are decoded by OSD. The decoding estimates of mDDs are used for voting the estimated LLR of $\mathcal{C}$, i.e., $\tilde{\mathbf{L}}$. The hard decision estimate $\hat{c} = [\hat{c}_{\alpha^i}, i \in I]$ is given by $\hat{c}_{\alpha^i} = \mathbb{1}[\tilde{L}_{\alpha^i} < 0]$.

3) Detection of codeword: Let $\mathbf{H}$ denote the parity-check matrix of $\mathcal{C}$. If $\mathbf{H}\hat{c}^T = \mathbf{0}$ is satisfied, it outputs the hard decision estimate $\hat{c}$ which is a codeword in $\mathcal{C}$. Here, T denotes the transpose. Otherwise, the process proceeds to the detection of decoding trap.

4) Detection of decoding trap: Let $\mathbf{H}_A$ denote the parity-check matrix of cDA. If $\mathbf{H}_A\hat{c}^T = \mathbf{0}$ but $\mathbf{H}\hat{c}^T \neq \mathbf{0}$, it indicates that the hard decision estimate $\hat{c}$ belongs to its cDA $\mathcal{A}(\mathcal{D}(\mathcal{C}))$ but not to the original code $\mathcal{C}$. In this case, the decoding process has fallen into a decoding trap, and the original LLR vector needs to be perturbed.

In particular, PDD-OSD takes the LLR vector $\mathbf{L}$ as input and operates iteratively. It contains two layers of loop: an inner loop and an outer loop. In the decoding process, the inner loop is executed to perform DD-OSD, the detection of legal codeword and the detection of decoding traps, while the outer loop handles the introduction of perturbations. If DD-OSD votes for a non-legal codeword and converges to a decoding trap, the decoding process moves to the outer loop, where perturbations are introduced to the original LLR vector $\mathbf{L}$.

In the first outer loop, no operation is performed before entering the inner loop; the process directly enters the inner loop to perform DD-OSD with input $\mathbf{L}$. In the second outer

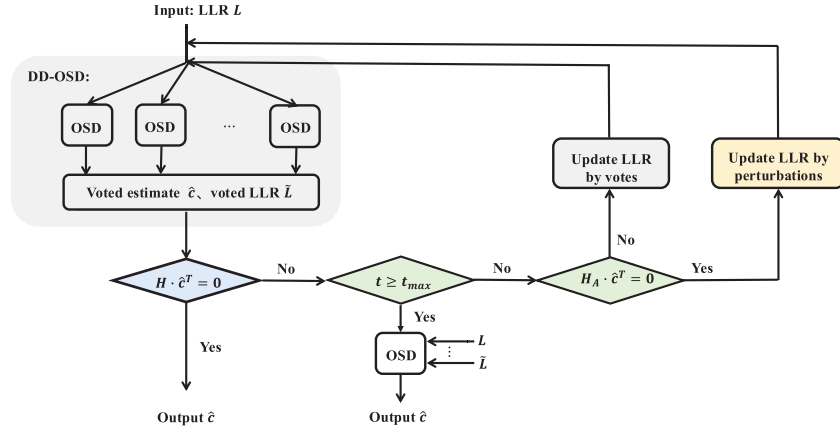


Fig. 2. The flowchart of PDD-OSD.

loop, the perturbed LLR vector $\tilde{L}$ is used as the new input to the inner loop, and DD-OSD is executed again with input $\tilde{L}$. In the next loops, the decoding process repeats as in the previous loops, until the codeword detection condition $H\hat{c}^T = 0$ is satisfied or the maximum number of outer loops is reached. In particular, after each outer loop decoding failure, OSD with order-1 is applied to the output of that loop to ensure that the result is a valid codeword $\hat{c}^{(t)}$. These candidate codewords $\{\hat{c}^{(0)}, \hat{c}^{(1)}, \dots, \hat{c}^{(t_{\max}-1)}\}$ are stored for subsequent selection. If all inner and outer loops do not satisfy the codeword detection condition, the final step selects the candidate codeword that is closest to the original received LLR vector L as the final output.

In summary, we provide the pseudo code of PDD-OSD in Algorithm 1.

Let t denote the number of outer loops and w denote the number of inner loops, respectively. The maximum number of inner loops is denoted by $w_{\max}$. The maximum number of outer loops, denoted by $t_{\max}$, also represents the number of perturbations. Let G_m denote the generator matrix of mDD.

In Line 2, the perturbations are introduced with $\mathcal{I}_0, \mathcal{I}_1, \dots, \mathcal{I}_{t_{\max}-1}$ when $t > 0$. In Lines 3-17, DD-OSD, the detection of codeword and the detection of decoding trap are executed as the inner loop, respectively. In Line 5, the derivative LLR vector L^β of the mDD in the direction β is calculated based on (2). In Line 6, the mDD in the direction β is decoded by OSD with order l . We denote the decoding process of OSD by $\text{OSD}(L^\beta, G_m, l)$. In Line 7, the soft votes from the direction β are obtained based on (3). In Line 9, the estimate of the original code $\mathcal{C}$ is voted by these soft votes from the directions set B based on (4). And the hard decision estimate $\hat{c}$ is obtained based on the indicator function $\mathbb{1}[\cdot]$. In Lines 10-11, if the hard decision estimate $\hat{c}$ is a codeword of original code, PDD-OSD ends the decoding process and returns the final output $\hat{c}$. In Line 12, if the decoder is detected to fall into a decoding trap, the inner loop is terminated. Otherwise, update the LLR vector based on soft votes to serve as the new input for the next inner loop. In Line 18, after each outer loop, OSD with order-1 is applied to generate a candidate codeword $\hat{c}^{(t)}$. The decoding process proceeds until

Algorithm 1 PDD-OSD

Input: The maximum outer and inner loop iterations $t_{\max}$, $w_{\max}$; the LLR L ; the index sets $\mathcal{I}_0, \mathcal{I}_1, \dots, \mathcal{I}_{t_{\max}-1}$; the factor λ ; the direction set B ; the generator matrix G_m , G ; the order l ; the parity-check matrix H , H_A

Output: Decoded codeword $\hat{c}$

```

1: for  $t = 0, \dots, t_{\max}$  do
2:    $L' \leftarrow \text{Perturbation}(L, \mathcal{I}_{t-1}, \lambda)$  if  $t > 0$ , else  $\tilde{L} \leftarrow L$ 
3:   for  $w = 0, \dots, w_{\max} - 1$  do
4:     if  $\beta \in B$  then
5:       Get  $L^\beta$  based on (2) with  $\tilde{L}$ 
6:        $\tilde{c}^\beta \leftarrow \text{OSD}(L^\beta, G_m, l)$ 
7:       Get  $\tilde{L}^\beta$  based on (3)
8:     end if
9:   Get  $\tilde{L}$  and  $\hat{c}$  based on (4) and indicator function  $\mathbb{1}[\cdot]$ 
10:  if  $H\hat{c}^T = 0$  then
11:    return  $\hat{c}$ 
12:  else if  $H_A\hat{c}^T = 0$  then
13:    Break
14:  else
15:     $\tilde{L} \leftarrow \tilde{L}$ 
16:  end if
17: end for
18:  $\hat{c}^{(t)} \leftarrow \text{OSD}(\tilde{L}, G, 1)$ 
19: end for
20: if  $H\hat{c}^T \neq 0$  then
21:    $\hat{c}^{(t_{\max})} \leftarrow \text{OSD}(L, G, 1)$ 
22:    $\hat{c} \leftarrow \underset{\hat{c}^{(t)} \in \{\hat{c}^{(0)}, \hat{c}^{(1)}, \dots, \hat{c}^{(t_{\max})}\}}{\text{argmax}} \sum_{i \in I} \hat{c}_{\alpha^i}^{(t)} \cdot L_{\alpha^i}$ 
23: end if
24: return  $\hat{c}$ 

```

it outputs a codeword or the maximum number of outer loops is reached. In Lines 20-22, if the detection of codeword is not satisfied after inner and outer loops, the candidate $\hat{c}^{(t)}$ that is the closest match to the received vector is selected as the final output $\hat{c}$.

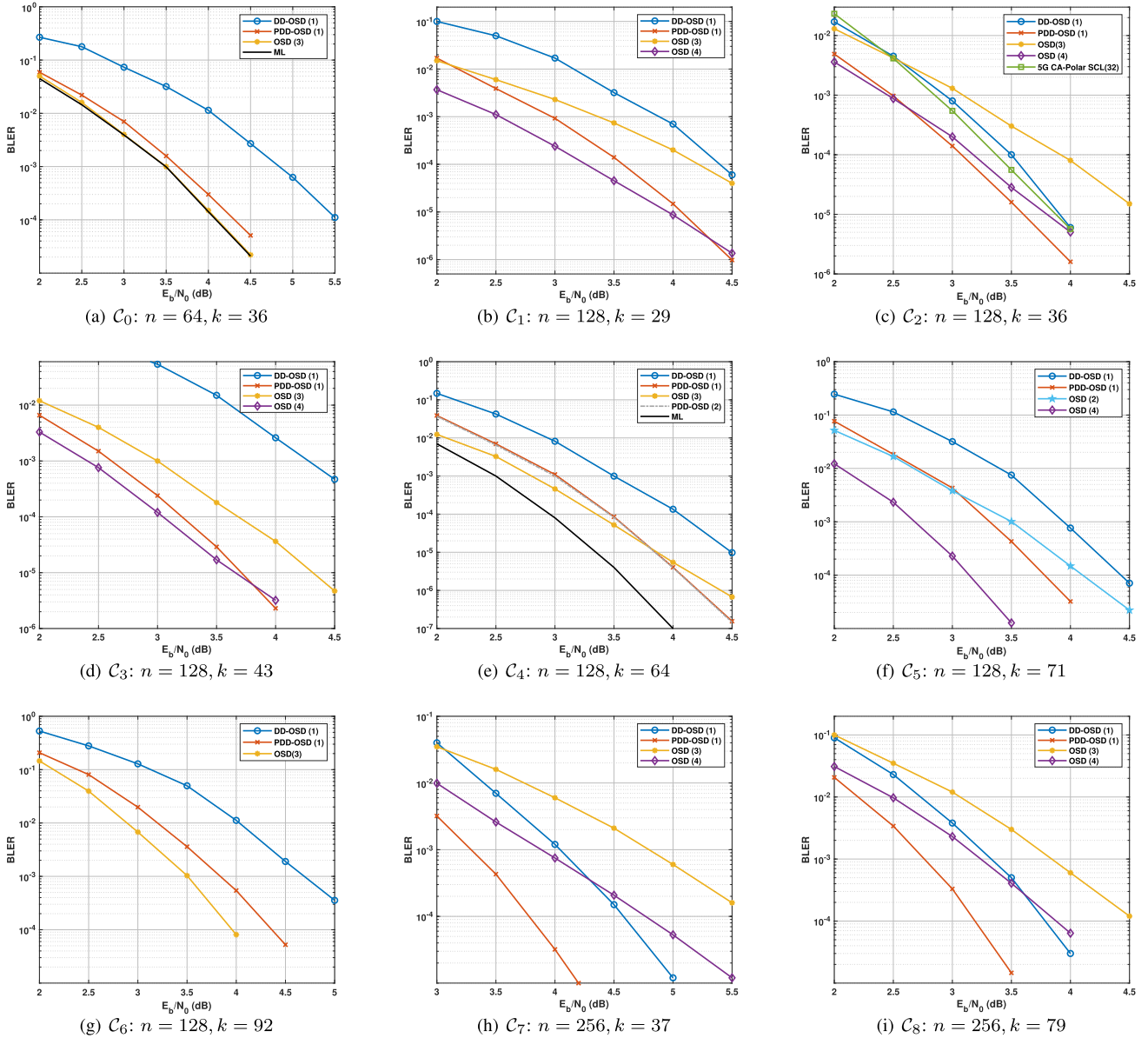


Fig. 3. Performance of decoding C_0 , C_1 , C_2 , C_3 , C_4 , C_5 , C_6 , C_7 and C_8 using PDD-OSD, DD-OSD and OSD.

IV. SIMULATIONS AND COMPLEXITY ANALYSIS

In this section, we provide the simulation results and the complexity analysis of PDD-OSD, respectively. DD-OSD, OSD and *cyclic redundancy check aided successive cancellation list* (CA-SCL) decoding are included as benchmarks to compare performance and complexity.

A. Simulations

Consider transmitting a codeword of $\mathcal{C}(n, k)$ over an AWGN channel using BPSK modulation. We denote the OSD, DD-OSD and PDD-OSD of order- l by OSD (l), DD-OSD (l) and PDD-OSD (l) with order $l = 1, 2, 3, 4$, respectively. Unless noted otherwise, we set $w_{\max} = 4$ and $\lambda = 0.5$.

Example 3: Consider the $(64, 36)$ eBCH code C_0 . The dimension of mDDs is $k_{\mathcal{M}} = 16$. We introduce 4 different

perturbations with $t_{\max} = 4$ and the index sets: $\mathcal{I}_0 = [40 : 45]$, $\mathcal{I}_1 = [35 : 40]$, $\mathcal{I}_2 = [30 : 35]$, and $\mathcal{I}_3 = [25 : 30]$. The simulation results are shown in Fig. 3(a). It shows that PDD-OSD (1) outperforms DD-OSD (1) by about 1.2 dB at the BLER of 10^{-4} . The gap between the performance of PDD-OSD (1) and that of ML is about 0.2 dB.

Example 4: Consider the $(128, 29)$ eBCH code C_1 . The dimension of mDDs is $k_{\mathcal{M}} = 14$. We introduce 5 different perturbations with $t_{\max} = 5$ and the index sets: $\mathcal{I}_0 = [65 : 75]$, $\mathcal{I}_1 = [55 : 65]$, $\mathcal{I}_2 = [45 : 55]$, $\mathcal{I}_3 = [35 : 45]$, and $\mathcal{I}_4 = [25 : 35]$. The simulation results are shown in Fig. 3(b). It shows that PDD-OSD (1) outperforms DD-OSD (1) by about 0.8 dB at the BLER of 10^{-4} .

Example 5: Consider the $(128, 36)$ eBCH code C_2 . The dimension of mDDs is $k_{\mathcal{M}} = 14$. We introduce 4 different perturbations with $t_{\max} = 4$ and the index sets: $\mathcal{I}_0 = [65 : 75]$,

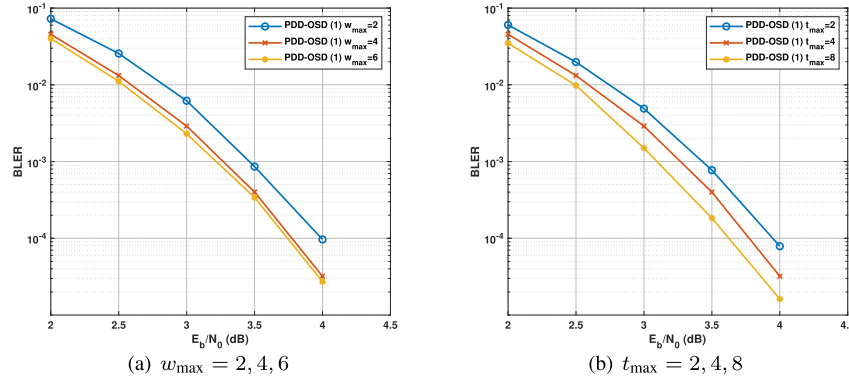


Fig. 4. Performance of PDD-OSD (1) under different $w_{\max}$ and $t_{\max}$.

$\mathcal{I}_1 = [55 : 65]$, $\mathcal{I}_2 = [40 : 55]$, and $\mathcal{I}_3 = [30 : 40]$. The simulation results are shown in Fig. 3(c). It shows that PDD-OSD (1) outperforms DD-OSD (1) and OSD (4) by about 0.3 dB and 0.2 dB at the BLER of 10^{-5} , respectively. For the polar code with the same parameter settings, PDD-OSD (1) can also outperform the CA-SCL decoding with a list size of 32 and cyclic redundancy check of length 8 by about 0.3 dB at the BLER of 10^{-5} .

Example 6: Consider the (128, 43) extended cyclic code $\mathcal{C}_3$. The exponents of the roots of its generator polynomial are 1, 3, 5, 7, 9, 11, 13, 19, 21, 27, 29, 43. The dimension of mDDs is $k_{\mathcal{M}} = 21$. We introduce 4 different perturbations with $t_{\max} = 4$ and the index sets: $\mathcal{I}_0 = [75 : 85]$, $\mathcal{I}_1 = [65 : 75]$, $\mathcal{I}_2 = [50 : 65]$, and $\mathcal{I}_3 = [40 : 50]$. The simulation results are shown in Fig. 3(d). It shows that PDD-OSD (1) outperforms DD-OSD (1) by about 1.6 dB at the BLER of 10^{-3} .

Example 7: Consider the (128, 64) eBCH code $\mathcal{C}_4$. The dimension of mDDs is $k_{\mathcal{M}} = 29$. We introduce 4 different perturbations with $t_{\max} = 4$ and the index sets: $\mathcal{I}_0 = [80 : 90]$, $\mathcal{I}_1 = [70 : 80]$, $\mathcal{I}_2 = [60 : 70]$, and $\mathcal{I}_3 = [50 : 60]$. The simulation results are shown in Fig. 3(e). It shows that PDD-OSD (1) outperforms DD-OSD (1) by about 0.65 dB at the BLER of 10^{-5} . The gap between the performance of PDD-OSD (1) and that of MLD is about 0.55 dB. It also shows that increasing the order does not lead to performance improvement of PDD-OSD. In DD-OSD, the final decision is determined by majority voting over the OSD decoding results of DDs. Consequently, even if all DDs are correctly decoded by OSD, erroneous votes may still arise during the majority voting process due to the impact of the received LLRs. This limits the performance of higher-order DD-OSD, and l-MLD errors do as well.

Example 8: Consider the (128, 71) eBCH code $\mathcal{C}_5$. The dimension of mDDs is $k_{\mathcal{M}} = 35$. We introduce 4 different perturbations with $t_{\max} = 4$ and the index sets: $\mathcal{I}_0 = [80 : 90]$, $\mathcal{I}_1 = [70 : 80]$, $\mathcal{I}_2 = [60 : 70]$, and $\mathcal{I}_3 = [50 : 60]$. The simulation results are shown in Fig. 3(f). It shows that PDD-OSD (1) outperforms DD-OSD (1) by about 0.6 dB at the BLER of 10^{-4} . The gap between the performance of PDD-OSD (1) and that of OSD (4) is about 0.6 dB.

Example 9: Consider the (128, 92) eBCH code $\mathcal{C}_6$. The dimension of mDDs is $k_{\mathcal{M}} = 42$. We introduce 4 different perturbations with $t_{\max} = 4$ and the index sets: $\mathcal{I}_0 = [90 : 100]$,

$\mathcal{I}_1 = [80 : 90]$, $\mathcal{I}_2 = [70 : 80]$, and $\mathcal{I}_3 = [60 : 70]$. The simulation results are shown in Fig. 3(g). It shows that PDD-OSD (1) outperforms DD-OSD (1) by about 0.7 dB at the BLER of 10^{-3} . The gap between the performance of PDD-OSD (1) and that of OSD (3) is about 0.4 dB at the BLER of 10^{-4} .

Example 10: Consider the (256, 37) eBCH code $\mathcal{C}_7$. The dimension of mDDs is $k_{\mathcal{M}} = 16$. We introduce 4 different perturbations with $t_{\max} = 4$ and the index sets: $\mathcal{I}_0 = [100 : 110]$, $\mathcal{I}_1 = [90 : 100]$, $\mathcal{I}_2 = [80 : 90]$, and $\mathcal{I}_3 = [70 : 80]$. The simulation results are shown in Fig. 3(h). It shows that PDD-OSD (1) outperforms DD-OSD (1) and OSD (4) by about 0.8 dB and 1.3 dB at the BLER of 10^{-5} , respectively.

Example 11: Consider the (256, 79) eBCH code $\mathcal{C}_8$. The dimension of mDDs is $k_{\mathcal{M}} = 31$. We introduce 4 different perturbations with $t_{\max} = 4$ and the index sets: $\mathcal{I}_0 = [120 : 130]$, $\mathcal{I}_1 = [110 : 120]$, $\mathcal{I}_2 = [100 : 110]$, and $\mathcal{I}_3 = [90 : 100]$. The simulation results are shown in Fig. 3(i). It shows that PDD-OSD (1) outperforms DD-OSD (1) and OSD (4) by about 0.6 dB and 0.7 dB at the BLER of 10^{-4} , respectively.

Then, we provide more simulation results under different parameter selections.

Consider the (256, 37) eBCH code. In Fig. 4(a), we fix $t_{\max} = 4$, and present the simulations of $w_{\max} = 2, 4, 6$. In Fig. 4(b), we fix $w_{\max} = 4$, and present the simulations of $t_{\max} = 2, 4, 8$. When $t_{\max} = 2$, we introduce 2 different perturbations with index sets: $\mathcal{I}_0 = [100 : 110]$ and $\mathcal{I}_1 = [90 : 100]$. When $t_{\max} = 4$, we introduce 4 different perturbations with index sets: $\mathcal{I}_0 = [100 : 110]$, $\mathcal{I}_1 = [90 : 100]$, $\mathcal{I}_2 = [80 : 90]$, and $\mathcal{I}_3 = [70 : 80]$. When $t_{\max} = 8$, we introduce 8 different perturbations with index sets: $\mathcal{I}_0 = [100 : 110]$, $\mathcal{I}_1 = [90 : 100]$, $\mathcal{I}_2 = [80 : 90]$, $\mathcal{I}_3 = [70 : 80]$, $\mathcal{I}_4 = [60 : 70]$, $\mathcal{I}_5 = [50 : 60]$, $\mathcal{I}_6 = [40 : 50]$, and $\mathcal{I}_7 = [30 : 40]$. It shows that increasing $t_{\max}$ and $w_{\max}$ leads to improved decoding performance. And the gain from increasing $w_{\max}$ from 2 to 4 is larger than that from 4 to 6.

Consider the (128, 29) eBCH code. In Fig. 5(a) we set different perturbations with $t_{\max} = 4$ and $t_{\max} = 2$. Specifically, in Perturbation A, we set $\mathcal{I}_0 = [65 : 75]$, $\mathcal{I}_1 = [55 : 65]$, $\mathcal{I}_2 = [45 : 55]$, and $\mathcal{I}_3 = [35 : 45]$; in Perturbation B, we set $\mathcal{I}_0 = [72 : 75]$, $\mathcal{I}_1 = [69 : 72]$, $\mathcal{I}_2 = [66 : 69]$, and $\mathcal{I}_3 = [63 : 66]$; in Perturbation C, we set $\mathcal{I}_0 = [55 : 75]$ and $\mathcal{I}_2 = [35 : 55]$. In Fig. 5(b), we present the simulation

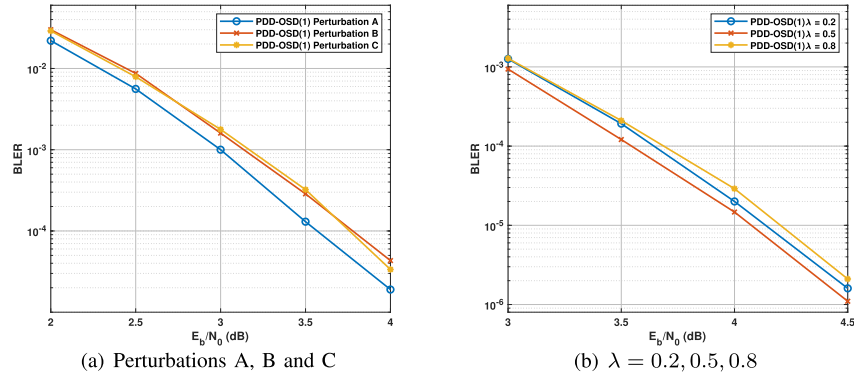


Fig. 5. Performance of PDD-OSD (1) under different perturbations and perturbation factors.

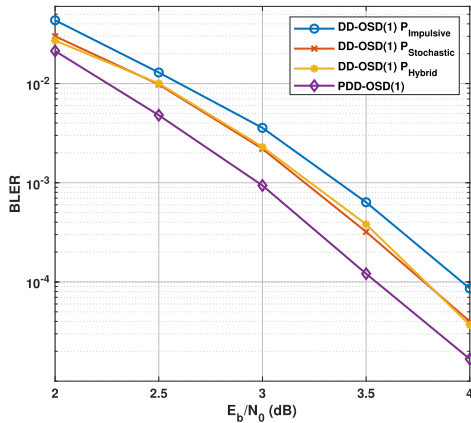


Fig. 6. The performance of different perturbation schemes.

results for $\lambda = 0.2, 0.5, 0.8$ while fixing all other parameters in Example 4. It shows that PDD-OSD (1) with Perturbation A outperforms that of Perturbations B and C, and PDD-OSD (1) with $\lambda = 0.5$ provides better performance among the considered values. In particular, applying perturbations to more bits leads to better performance with the same number of perturbations. And more frequent perturbations acting on fewer bits achieve better performance than a small number of perturbations covering many bits with the same total number of perturbed bits. Different selections of the number and width of perturbations result in performance variations, providing a trade-off between trap escape and decoding complexity.

At the end of the subsection, we further extend our investigation to other state-of-the-art decoding algorithms with perturbation schemes [31], [32], [33], [34]. These perturbation schemes have been shown to be effective for low-density parity-check codes and polar codes. Specifically, an impulsive perturbation scheme for *belief propagation* (BP) decoding [31] perturbs a small number of the least reliable received symbols in the opposite direction with a large fixed magnitude. In [32] and [33], stochastic perturbation methods are proposed for *successive cancellation* (SC) decoding, either by injecting Gaussian noise or by exploiting the sequential intermediate computations and decision updates inherent in SC decoding. In [34], a hybrid perturbation-based SC decoding scheme for long polar codes is proposed, which first applies random

TABLE II
FLOP COUNTS OF EACH MAJOR STEP PER ITERATION

Decoding process	FLOPs
Perturbing	0.896k
Calculating derivatives	16.256k
Re-encoding derivatives	144.018k
Voting	16.256k

perturbation and then triggers biased perturbation on disagree bits according to *cyclic redundancy check* (CRC) feedback. This scheme achieves excellent performance comparable to that of CRC-aided SC list decoding. As a result, we further investigate three adapted perturbation schemes within the proposed framework, including impulsive perturbation, stochastic perturbation, and hybrid perturbation. We denote them as $P_{\text{Impulsive}}$, $P_{\text{Stochastic}}$ and P_{Hybrid} , respectively.

Consider the (128, 29) eBCH code. For the perturbation scheme $P_{\text{Impulsive}}$, we introduce 5 different perturbations with $t_{\max} = 5$ and the index sets: $\mathcal{I}_0 = [65 : 75]$, $\mathcal{I}_1 = [55 : 65]$, $\mathcal{I}_2 = [45 : 55]$, $\mathcal{I}_3 = [35 : 45]$ and $\mathcal{I}_4 = [25 : 35]$. The LLRs of the perturbed bits are forced to 20 with reversed signs. For the perturbation scheme $P_{\text{Stochastic}}$, we introduce 5 different perturbations with $t_{\max} = 5$, where the standard deviations of the Gaussian noise are set to 0.0125, 0.0250, 0.0375, 0.0500 and 0.0625, respectively. For the perturbation scheme P_{Hybrid} , we also introduce 5 different perturbations with $t_{\max} = 5$. Once a decoding trap is detected, Gaussian noise is first injected to escape the local optimum; if two consecutive detected codewords are identical in the subsequent decoding attempts, biased perturbation is then applied based on the disagree bits. The disagree bits are selected according to the method in [34]. The simulation results show that PDD-OSD (1) outperforms DD-OSD (1) with the other three perturbation schemes.

B. Complexity Analysis

In terms of complexity, we analyze both the average and worst cases for PDD-OSD. The *floating point operations* (FLOPs) cost is $|B|(2n + C_d)$ per inner loop, where the cost of a single derivative direction is given by

TABLE III
NUMERICAL COMPLEXITY RESULTS FOR (128, 36) EBCH CODE AT
DIFFERENT E_b/N_0

Algorithm	E_b/N_0	$\bar{C}_{ave}$	$\bar{C}_{iter}$	$\bar{I}_{ave}$
PDD-OSD	2 dB	371.1k	177.0k	2.097
	3 dB	224.7k	177.2k	1.268
DD-OSD	2 dB	308.9k	176.5k	1.75
	3 dB	221.7k	176.5k	1.256

$C_d = (n/2)\log_2(n/2) + (n/2 - k_{\mathcal{M}})\sum_{i=0}^l \binom{k_{\mathcal{M}}}{i}$. Here, the complexity of calculating derivatives and voting is $2n|B|$ per inner loop. The asymptotic FLOPs complexity of PDD-OSD (1) is $O((t_{\max} + 1)|B|n^2)$ in the worst case, while that of DD-OSD (1) is $O(|B|n^2)$. Specifically, the asymptotic complexity of the perturbing, calculating derivative, and voting steps is $O(|B|n)$. And the FLOPs of the major step in PDD-OSD (1) per iteration are given in Table II. It shows that, for the (128, 36) eBCH code, the overall computational FLOPs cost is dominated by re-encoding derivatives, while the costs of perturbing, calculating derivatives, and voting are relatively minor.

First, consider the average-case complexity. The average complexity of PDD-OSD is slightly higher than that of DD-OSD. For the (128, 36) eBCH code, the cost of PDD-OSD (1) is 198.2k FLOPs at $E_b/N_0 = 3.5$ dB on average. Its average number of inner loops and outer loops is 1.1223 and 1.000265, respectively. The overall average number of iterations is 1.1229. For comparison, DD-OSD (1) costs 198.0k FLOPs on average. Its average number of loops is 1.122. For the (256, 37) eBCH code, the cost of PDD-OSD (1) is 1,418.0k FLOPs at $E_b/N_0 = 4$ dB on average. Its average number of inner loops and outer loops is 1.673 and 1.003212, respectively. The overall average number of iterations is 1.679. For comparison, DD-OSD (1) costs 1,411.3k FLOPs on average. Its average number of loops is 1.671.

Moreover, the average-case comparison with DD-OSD at different E_b/N_0 is given in Table III. Specifically, let $\bar{C}_{ave}$ denote the average decoding complexity, let $\bar{I}_{ave}$ denote the average number of decoding iterations, and let $\bar{C}_{iter}$ denote the average complexity per iteration. Table III shows that the average number of decoding iterations increases at low E_b/N_0 , leading to a higher average decoding complexity. And the average decoding complexity of PDD-OSD at low E_b/N_0 is higher than that of DD-OSD, whereas this gap becomes much smaller at moderate to high E_b/N_0 .

Then, consider the worst-case complexity. When $w_{\max}$ is the maximum number of iterations of DD-OSD, the worst-case computational complexity of PDD-OSD equals $(t_{\max} + 1)$ times the complexity of DD-OSD, along with $(t_{\max} + 2)$ additional executions of OSD with order-1. Specifically, the worst-case computational complexity of DD-OSD is $w_{\max}|B|(2n + C_d)$. For the (128, 36) eBCH code, the worst-case cost of PDD-OSD (1) and DD-OSD (1) is 3,557k and 706.1k FLOPs, respectively. For comparison, OSD (4) and OSD (3) require 6,137k and 718k FLOPs, respectively. For the (256, 37) eBCH code, the worst-case cost of PDD-OSD (1)

and DD-OSD (1) is 16,955k and 3,378k FLOPs, respectively. For comparison, OSD (4) and OSD (3) require 16,321k and 1,857k FLOPs, respectively.

In addition, skipping rules [1], [13], [14], [35] can help further reduce the number of TEPs in OSD. We compare the number of TEPs of the PDD-OSD with that of the OSD algorithm with skipping rules [1]. For the (128, 36) eBCH code, the maximum number of TEPs is 66,712 and the average number of TEPs is 30,105 in OSD (4). For comparison, PDD-OSD (1) outperforms OSD (4) by about 0.2 dB. The maximum number of TEPs is 38,100 with $t_{\max} = 4$ and $w_{\max} = 4$ in PDD-OSD (1). Moreover, skipping rules can also be incorporated into the PDD-OSD framework.

V. CONCLUSION

To conclude, the main contributions of the paper are

- The l -MLD errors in derivative directions may trap DD-OSD, and such decoding traps are detected based on cDA.
- Perturbations help escape from the current trapped decoding and further avoid the traps by moving errors out of MRIPs in mDDs.
- A probability analysis is provided to theoretically evaluate the error probability of perturbed mDD.

Furthermore, more advanced perturbation strategies and parameter selection methods may further improve the performance of PDD-OSD, which we will leave for future investigation.

REFERENCES

- [1] M. P. C. Fossorier and S. Lin, "Soft-decision decoding of linear block codes based on ordered statistics," *IEEE Trans. Inf. Theory*, vol. 41, no. 5, pp. 1379–1396, Sep. 1995.
- [2] A. Valembois and M. Fossorier, "Box and match techniques applied to soft-decision decoding," *IEEE Trans. Inf. Theory*, vol. 50, no. 5, pp. 796–810, May 2004.
- [3] W. Jin and M. P. C. Fossorier, "Reliability-based soft-decision decoding with multiple biases," *IEEE Trans. Inf. Theory*, vol. 53, no. 1, pp. 105–120, Jan. 2007.
- [4] C. Yue, M. Shirvanimoghaddam, G. Park, O.-S. Park, B. Vucetic, and Y. Li, "Probability-based ordered-statistics decoding for short block codes," *IEEE Commun. Lett.*, vol. 25, no. 6, pp. 1791–1795, Jun. 2021.
- [5] Y. Wu and C. N. Hadjicostis, "Soft-decision decoding of linear block codes using preprocessing and diversification," *IEEE Trans. Inf. Theory*, vol. 53, no. 1, pp. 378–393, Jan. 2007.
- [6] C. Yue, M. Shirvanimoghaddam, B. Vucetic, and Y. Li, "A revisit to ordered statistics decoding: Distance distribution and decoding rules," *IEEE Trans. Inf. Theory*, vol. 67, no. 7, pp. 4288–4337, Jul. 2021.
- [7] X. Li, W. Chen, L. Chen, Y. Li, and H. Zhang, "Order skipping ordered statistics decoding and its performance analysis," in *Proc. IEEE Inf. Theory Workshop (ITW)*, Nov. 2024, pp. 448–453.
- [8] M. P. Fossorier and S. Lin, "Reliability-based information set decoding of binary linear block codes," *IEICE Trans. Fundamentals*, vol. 82, no. 10, pp. 2034–2042, Oct. 1999.
- [9] M. Bossert, R. Schulz, and S. Bitzer, "On hard and soft decision decoding of BCH codes," *IEEE Trans. Inf. Theory*, vol. 68, no. 11, pp. 7107–7124, Nov. 2022.
- [10] Y. Wang, J. Liang, Q. Wang, and X. Ma, "Representative ordered statistics decoding of staircase matrix codes," *IEEE Trans. Commun.*, vol. 73, no. 4, pp. 2148–2158, Apr. 2025.
- [11] S. Yu, B. Zhang, and Q. Huang, "Ordered statistics derivative decoding for affine-invariant codes without Gaussian elimination," *IEEE Trans. Commun.*, vol. 73, no. 11, pp. 11006–11016, Nov. 2025.
- [12] C. Yue, M. Shirvanimoghaddam, G. Park, O.-S. Park, B. Vucetic, and Y. Li, "Linear-equation ordered-statistics decoding," *IEEE Trans. Commun.*, vol. 70, no. 11, pp. 7105–7123, Nov. 2022.

- [13] Y. Wang, J. Liang, and X. Ma, "Local constraint-based ordered statistics decoding for short block codes," in *Proc. IEEE Inf. Theory Workshop (ITW)*, Nov. 2022, pp. 107–112.
- [14] J. Liang, Y. Wang, S. Cai, and X. Ma, "A low-complexity ordered statistic decoding of short block codes," *IEEE Commun. Lett.*, vol. 27, no. 2, pp. 400–403, Feb. 2023.
- [15] Q. Huang and S. Yu, "Perturbed derivative decoding based on ordered statistics for cyclic codes," in *Proc. IEEE Inf. Theory Workshop (ITW)*, Sep. 2025, pp. 590–595.
- [16] C. Choi and J. Jeong, "Fast soft decision decoding algorithm for linear block codes using permuted generator matrices," *IEEE Commun. Lett.*, vol. 25, no. 12, pp. 3775–3779, Dec. 2021.
- [17] Q. Wang, Y. Wang, Y. Wang, J. Liang, and X. Ma, "Random staircase generator matrix codes: Coding theorem, performance analysis, and code design," *IEEE Trans. Inf. Theory*, vol. 71, no. 5, pp. 3497–3509, May 2025.
- [18] Q. Huang and B. Zhang, "Minimal derivative descendants of cyclic codes," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Jun. 2023, pp. 286–291.
- [19] Q. Huang and B. Zhang, "Derivative descendants of cyclic codes and derivative decoding," *IEEE Trans. Inf. Theory*, vol. 70, no. 4, pp. 2395–2410, Apr. 2024.
- [20] L. Deng et al., "Perturbed adaptive belief propagation decoding for high-density parity-check codes," *IEEE Trans. Commun.*, vol. 69, no. 4, pp. 2065–2079, Apr. 2021.
- [21] A. Hocquenghem, "Codes correcteurs d'erreurs," *Chiffres*, vol. 2, pp. 147–156, Sep. 1959.
- [22] R. C. Bose and D. K. Ray-Chaudhuri, "On a class of error correcting binary group codes," *Inf. Control*, vol. 3, no. 1, pp. 68–79, Mar. 1960.
- [23] I. Reed, "A class of multiple-error-correcting codes and the decoding scheme," *Trans. IRE Prof. Group Inf. Theory*, vol. 4, no. 4, pp. 38–49, Sep. 1954.
- [24] D. E. Müller, "Application of Boolean algebra to switching circuit design and to error detection," *Trans. IRE Prof. Group Electron. Comput.*, vol. 3, no. 3, pp. 6–12, Sep. 1954.
- [25] M. Legeay, "Permutation decoding: Towards an approach using algebraic properties of the-subcode," in *Proc. Workshop Coding Cryptogr. (WCC)*, Apr. 2011, pp. 193–202.
- [26] H. F. Mattson and G. Solomon, "A new treatment of bose-chaudhuri codes," *J. Soc. for Ind. Appl. Math.*, vol. 9, no. 4, pp. 654–669, Dec. 1961.
- [27] F. J. MacWilliams and N. J. A. Sloane, *The Theory of Error-Correcting Codes*, vol. 16. Amsterdam, The Netherlands: Elsevier, 1977.
- [28] S. Lin and D. J. Costello, *Error Control Coding*, 2nd ed., Upper Saddle River, NJ, USA: Prentice-Hall, 2004.
- [29] W. Ryan and S. Lin, *Channel Codes: Classical and Modern*. Cambridge, U.K.: Cambridge Univ. Press, 2009.
- [30] S.-Y. Chung, T. J. Richardson, and R. L. Urbanke, "Analysis of sum-product decoding of low-density parity-check codes using a Gaussian approximation," *IEEE Trans. Inf. Theory*, vol. 47, no. 2, pp. 657–670, Feb. 2001.
- [31] H. Lee, Y.-S. Kil, M. Jang, S.-H. Kim, O.-S. Park, and G. Park, "Multi-round belief propagation decoding with impulsive perturbation for short LDPC codes," *IEEE Wireless Commun. Lett.*, vol. 9, no. 9, pp. 1491–1494, Sep. 2020.
- [32] A. Çagri Arli and O. Gazi, "Noise-aided belief propagation list decoding of polar codes," *IEEE Commun. Lett.*, vol. 23, no. 8, pp. 1285–1288, Aug. 2019.
- [33] Z. Liu, L. Yao, S. Yuan, G. Yan, Z. Ma, and Y. Liu, "Performance analysis of perturbation-enhanced SC decoders," *IEEE Commun. Lett.*, vol. 29, no. 3, pp. 507–511, Mar. 2025.
- [34] Z. Yang, L. Chen, K. Qin, X. Wang, and H. Zhang, "Perturbation-based decoding schemes for long polar codes," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Jun. 2025, pp. 1–6.
- [35] Q. Wang, Y. Wang, J. Liang, L. Song, and X. Ma, "A two-stage soft-decision decoding algorithm for BCH codes," *IEEE Trans. Commun.*, vol. 73, no. 12, pp. 12979–12992, Dec. 2025.



Shuyan Yu received the B.S. degree in information countermeasure technology from Xidian University, Xi'an, China, in 2019, and the Ph.D. degree in electrical engineering from Beihang University, Beijing, China, in 2026. Her current research interests include error control codes and cryptography.



Tianyu Li received the B.S. degree in electrical engineering from Beihang University (BUAA), Beijing, China, in 2024, where he is currently pursuing the M.S. degree with the School of Electronic and Information Engineering. His research interests include error-control codes for communication systems.



Qin Huang (Senior Member, IEEE) received the B.E. and M.E. degrees in electrical engineering from Southeast University, Nanjing, China, in 2005 and 2007, respectively, and the Ph.D. degree in electrical engineering from the University of California, Davis, CA, USA, in 2011. He joined LinkA Media Devices Corporation, Santa Clara, CA, USA, in 2011. He is currently a Professor with the School of Electronic and Information Engineering, Beihang University (BUAA), Beijing, China. His research interests include classical and modern coding theory and signal processing and their applications on communications and storage systems. He served as an Associate Editor for IEEE TRANSACTIONS ON COMMUNICATIONS.